

Unveiling Vulnerabilities in Deep Learning-based Malware Detection: Differential Privacy Driven Adversarial Attacks

Rahim Taheri^{a,*}, Mohammad Shojafar^b, Farzad Arabikhan^a, Alexander Gegov^a

^aFaculty of Technology, School of Computing, University of Portsmouth, UK

^bInstitute for Communication Systems, 5G/6G Innovation Centre (5G/6GIC), University of Surrey, UK

Abstract

The exponential increase of Android malware creates a severe threat, motivating the development of machine learning and especially deep learning-based classifiers to detect and mitigate malicious applications. However, these classifiers are susceptible to adversarial attacks that manipulate input data to deceive the classifier and compromise performance. This paper investigates the vulnerability of deep learning-based Android malware classifiers against two adversarial attacks: Data Poisoning with Noise Injection (DP-NI) and Gradient-based Data Poisoning (GDP). In these attacks, we explore the utilization of differential privacy techniques by attackers aiming to compromise the effectiveness of deep learning based Android malware classifiers. We propose and evaluate a novel defense mechanism, Differential Privacy-Based Noise Clipping (DP-NC), designed to enhance the robustness of Android malware classifiers against these adversarial attacks. By leveraging deep neural networks and adversarial training techniques, DP-NC demonstrates remarkable efficacy in mitigating the impact of both DP-NI and GDP attacks. Through extensive experimentation on *three* diverse Android datasets (Drebin, Contagio, and Genome), we evaluate the performance of DP-NC against proposed adversarial attacks. Our results show that DP-NC significantly reduces the false-positive rate and improves classification accuracy across all datasets and attack scenarios. For instance, our findings on the Drebin dataset reveal a significant decrease in accuracy to 51% and 30% after applying DP-NI and GDP techniques, respectively. However, upon applying the DP-NC defense mechanism, the accuracy in both cases improved to approximately 70%. Furthermore, employing DP-NC defense against DP-NI and GDP attacks leads to a notable reduction in false positive rates by 45.46% and 7.67%, respectively. Similar results have been obtained in two other datasets, Contagio and Genome. These results underscore the effectiveness of DP-NC in enhancing the robustness of deep learning-based Android malware classifiers against adversarial attacks.

Keywords: Adversarial attacks, Android malware detection, Deep learning, Differential privacy, Gradient perturbation

1. Introduction

The increasing popularity of Android applications has made Android malware a significant threat to the security and privacy of mobile device users [1]. Developing effective Android malware detection models becomes important as malicious actors continually evolve their tactics. Machine learning (ML) and especially deep learning (DL)-based approaches have shown promise in this domain, achieving notable success in identifying and mitigating malware threats [2]. However, deploying such models raises concerns about susceptibility to adversarial attacks [3, 4]. Adversarial attacks against DL-based malware detection models have demonstrated the vulnerabilities of these systems. Researchers in this domain are working on developing novel methodologies that detect Android malware effectively and withstand the deliberate manipulation attempts by adversaries [5, 6].

Differential privacy [7] has emerged as a significant idea for improving privacy in ML by including noise into the training process to safeguard sensitive information in the training data. This technique provides formal assurances of privacy by assuring that the presence or absence of any single training sample has little influence on the model's output, reducing the risk of privacy breaches caused by unintended information leakage. By perturbing the training data with carefully calibrated noise, the differential privacy framework can cause an increase/decrease in the effectiveness of adversarial attacks [8, 9], thus controlling the robustness of ML models to malicious manipulation. This capability of differential privacy, which allows for adding minimal perturbation in the form of noise to the training data, can be used by an attacker to generate adversarial examples [10].

This paper addresses how attackers can use differential privacy to generate adversarial samples and compromise the robustness of the Android malware detection, system. We introduce innovative methods for generating adversarial examples using differential privacy principles. The innovation of this paper lies in the proposed attack methods aimed at perturbing data

*Corresponding author

Email addresses: rahim.taheri@port.ac.uk (Rahim Taheri), m.shojafar@surrey.ac.uk (Mohammad Shojafar), farzad.arabikhan@port.ac.uk (Farzad Arabikhan), alexander.gegov@port.ac.uk (Alexander Gegov)

with noise generated by differential privacy. Additionally, a defense method is proposed that utilizes differential privacy to defend against these types of attacks.

Our approaches aim to investigate differential privacy-based adversarial attacks by crafting malicious training data and using it to mislead classification models. So, we present two distinct attack methods: differential privacy-based noise injection and gradient-based differential privacy for model poisoning. Each method leverages specific differential privacy techniques to craft adversarial examples. Furthermore, we propose a noise-clipping method based on differential privacy, which eliminates potential adversarial samples and aids in fortifying the robustness of classification against adversarial attacks. As with any protective measures, we do not expect our defense mechanism to completely eliminate the impacts of attacks. However, it is expected to be very robust against adversarial attacks. The implementation code for this study is available at [11].

1.1. Contributions

In this paper, we make several contributions to the field of adversarial ML and Android malware detection, including:

- We investigate the potential of utilizing differential privacy-based techniques by attackers to generate adversarial samples and by defense systems to mitigate adversarial attacks.
- We propose novel adversarial attack methods, namely the Differential Privacy-based Noise Injection (DP-NI) Attack and the Gradient-based Data Poisoning (GDP) Attack. These attacks leverage differential privacy principles to generate adversarial samples that mislead deep neural networks. The DP-NI Attack introduces Laplace noise into the input data to create adversarial samples. The GDP Attack perturbs the gradients of the loss function during training to craft adversarial examples.
- We propose a comprehensive defense mechanism, DP-NC Defense (Differential Privacy-based Noise Clipping), designed to mitigate the impact of differential privacy-based adversarial attacks on binary classification tasks. Our defense strategy involves modifying the training objective and incorporating differential privacy noise into the gradient computation.
- We evaluate the effectiveness of proposed attack and defense methods through extensive experiments on benchmark Android malware datasets. We provide empirical evidence demonstrating the efficacy of attacks in generating adversarial samples and the robustness of defense method in mitigating the impact of proposed attacks and maintaining model accuracy.

1.2. Organization

This paper is organized as follows: Section 2 delves into the literature review in this study. In section 3, we explain this paper's threat model and data representation. Sections 4 and 5 provide detailed formulations, mathematical explanations, and

algorithmic descriptions of each proposed attack and defense method. We then evaluate the performance of these methods on benchmark datasets in sections 6 and 7. Section 8 discusses the results and limitations of the proposed methods. Finally, we conclude our work and outline potential future research directions in section 9.

2. Literature Review

This section provides an overview of the state-of-the-art approaches at the intersection of deep learning, Android malware detection, and privacy-preserving techniques.

The authors in [12] present a novel concept of scalable differential privacy in adversarial learning. This innovative approach lays the foundation for incorporating differential privacy principles into adversarial attacks to mislead classification models while ensuring privacy guarantees. Mathias et al. [13] explore the relationship between differential privacy and adversarial robustness in ML. This author et al. in [14] present a novel approach for achieving certified robustness to adversarial examples while integrating differential privacy. This methodology can be adapted to enhance the security of Android malware detection models.

In the Android malware detection area, Nurislam et al. [15] discuss the robustness threats associated with differential privacy, shedding light on potential vulnerabilities that adversaries may exploit in Android malware detection systems. Two types of research related to the proposed methods of this paper are presented in [16] and [17]. In [16], the paper highlights the threats posed by gradient masking in the context of differential privacy. Understanding these threats is crucial for crafting effective adversarial attacks. The concept of the heterogeneous Gaussian mechanism, which preserves differential privacy in deep learning, is presented in [17]. Zhu et al. [18] differentiated between privacy-preserving prediction and training solutions, offering insights into various techniques such as differential privacy, secure multi-party computation, and homomorphic encryption used in these areas. Their classification can serve as a reference for privacy preservation in IoT malware detection systems. Conversely, in [19], the authors delve into the defense aspect of differential privacy by proposing a method to defend against data inference attacks. It highlights the importance of privacy-preserving mechanisms in securing sensitive data. The authors in [20] provide a comprehensive view of differential privacy in deep learning, encompassing both privacy preservation and adversarial robustness. It offers a holistic perspective on the challenges and solutions in the domain. In [21], authors offer a unified perspective on differential privacy and robustness to adversarial examples. This paper explores the connections between these two critical aspects, which are essential for your research on privacy-driven adversarial attacks.

The above literature provides valuable insights into privacy in IoT environments and privacy-preserving solutions. Based on our best knowledge, there has not yet been any research about adversarial sample generation using differential privacy concept by attackers, and there is still a gap in this area. Our

Table 1: A list of ML-based approaches reviewed for differential privacy-based adversarial attacks and defenses (DP:=differential privacy,AD:=adversarial Setting)

Ref.	Method	Classification	Dataset	Application
[12]	Attack and Defense	DNN	MNIST and CIFAR	Managing trade-off between DP and AD
[19]	Defense	DNN	MNIST and CIFAR	Using DP as a part of defense method
[13]	Attack and Defense	DNN	MNIST and CIFAR	Using DP for providing robustness
[14]	Attack	CNN	MNIST and SVHN	Using DP for providing robustness
[22]	Attack and Defense	SVM	Network Data	Using DP as a part of defense method
[16]	Attack	DNN	MNIST	Providing privacy
[15]	Attack	CNN	MNIST and CIFAR	Robustness against threats of DP
[17]	Attack	DNN	MNIST and CIFAR	Preserving DP in Deep learning for robustness
This paper	Attack and Defense	DNN	IoT Android Dataset	Using DP for generating adversarial samples

research endeavors to bridge this gap. In the following sections, we provide detailed descriptions and evaluations of these methods, contributing to the growing knowledge at the intersection of deep learning, Android malware detection, and privacy preservation. Table 1 presents some works employing differential privacy in adversarial attacks and defenses.

3. Preliminaries

This section outlines this paper’s fundamental framework and preliminaries, including a threat model, data representation, differential privacy concept, and deep learning model.

3.1. Threat Model

Our threat model encompasses the following aspects:

1. We consider adversaries that are well-versed in DL and differential privacy. They are capable of generating differential privacy based adversarial samples that exploit model vulnerabilities. These adversaries may pursue poisoning objectives. They may attempt to poison the training data of malware detection models with malicious samples.
2. Privacy-focused adversaries know the differential privacy budget allocated to the model. They attempt to infer sensitive information from this budget and will use this information to generate adversarial samples.
3. Formally, we define the adversary’s objective as follows: Let $\mathcal{D}$ denote the training dataset, $\mathcal{M}$ represent the DL model, and $\mathcal{A}$ denote the adversary. The adversary seeks to maximize the model’s utility while minimizing the privacy loss incurred by injecting adversarial samples. Mathematically, the adversary’s objective can be formulated as equation 1:

$$\underset{\mathcal{A}}{\text{maximize}} \quad \text{Utility}(\mathcal{M}(\mathcal{D} \cup \mathcal{A})) - (\text{Privacy Loss}(\mathcal{A})) \quad (1)$$

Here, the utility function measures the model’s performance on benign and adversarial samples. In contrast, the privacy loss function quantifies the degree of information leakage about individual training samples due to the presence of generated adversarial samples.

3.2. Data Representation

In this paper, we focus on the static analysis of Android malware. We extract permissions from Android samples and construct a sparse matrix based on them to achieve this. In this matrix, the presence of a specific permission in a sample is represented by 1, while its absence is denoted by 0. If we consider n as the number of samples, each with m features, the data matrix can be written as equation 2:

$$D = \{(x_i, y_i) \mid \forall i = 1, \dots, n\}, \quad (2)$$

Here, x_i represents the vector representation corresponding to the i -th sample, with each component encapsulating the selected feature. The label associated with the feature is denoted as $y_i \in \{0, 1\}$. Furthermore, x_{ij} signifies the binary value of the j -th feature within the i -th sample, where $\forall j = 1, \dots, m$. Specifically, x_{ij} assumes a value of 1 if the j -th component exists within x_i , and 0 otherwise, effectively encoding the presence or absence of each feature. Notable notations include n as the total count of samples, and $X \subseteq \{0, 1\}^m$ representing an m -dimensional feature space. This comprehensive data representation forms the foundation for our proposed robust Android malware detection system.

3.3. Differential Privacy

Differential privacy is a foundational concept in data analysis for preserving privacy and is an essential part of our proposed method. It provides a framework for ensuring that the presence or absence of an individual’s data does not significantly affect the outcomes of computations [23, 24].

A mechanism $\mathcal{M}$ is said to satisfy ϵ -differential privacy if, for all datasets D and D' that differ in a single individual’s data point and for all subsets of possible outputs S :

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{M}(D') \in S] \quad (3)$$

where $\epsilon \geq 0$ is a privacy parameter. Smaller values of ϵ correspond to stronger privacy guarantees. When $\epsilon = 0$, it indicates perfect privacy [25, 26]. Differential privacy introduces two primary constraints:

1. Privacy Parameter (ϵ): This parameter quantifies privacy and utility trade-offs. A smaller ϵ implies stronger privacy guarantees but may result in less accurate data analysis.
2. Composition Theorem: For a sequence of data, the total privacy cost is bounded by the sum of the individual privacy parameters [27, 28].

3.4. Deep Learning Model

In this paper, we consider a simple DNN with Four layers, and its architecture is presented in Figure. 1.



Figure 1: DNN architecture

4. Proposed Attacks

This section explains our proposed methods for generating adversarial samples using differential privacy against deep neural networks. Our approaches aim to propose adversarial attacks by incorporating differential privacy principles, ensuring that crafted adversarial samples mislead classification models. We propose two distinct attack methods: "DP-NI Attack" and "GDP Attack". These attacks are presented in the following subsections.

4.1. DP-NI Attack

In this sub-section, we propose Data Poisoning with Noise Injection (DP-NI) as an adversarial attack that leverages differential privacy principles to generate adversarial samples by noise injection. The goal is to craft adversarial samples that lead to misclassification by the model.

Formal Definition: Let x be the original input data and y be the real label. The classification function is denoted as $f(x)$. Our objective is to create an adversarial sample x_{adv} that causes

Algorithm 1 DP-NI Attack: Data Poisoning with Noise Injection

Input: M :Model, $D = (x, y)$, ϵ :Privacy budget, s :Sensitivity, η : Learning rate
Output: x_{adv}

```

1:  $\sigma \leftarrow \frac{\epsilon}{s}$ 
2: for each  $i$  in  $N_{ep}$  do
3:   for each batch  $(x, y)$  in training data do
4:      $U_s = \text{Rand}(x.\text{shape})$ 
5:      $N = -\sigma \times \text{Sign}(U_s - 0.5) \times \log(1 - 2 \times |U_s - 0.5|)$ 
6:      $x_{adv} = x + N$ 
7:     if Check-Differential-Privacy( $x_{adv}, s, \epsilon$ ) then
8:        $y_{adv} \leftarrow M(x_{adv})$ 
9:        $\text{loss} \leftarrow \text{calculate\_loss}(y_{adv}, y)$ 
10:      Backpropagate-gradients()
11:      Update model parameters using an optimizer
12:     else
13:       Continue
14:     end if
15:   end for
16: end for
17: return  $x_{adv}$ 

```

the model to misclassify, while differential privacy is used to add noise and generate an adversarial sample.

In this attack method, the adversary generates noise using the Laplace distribution and then applies the differential privacy-based technique to add the generated noise to the training data. The steps for this process are as follows:

- Computing the noise scale (σ) based on the desired privacy parameter (ϵ) and the model's sensitivity. Subsequently, Laplace noise (N) generating from the Laplace distribution with scale σ , equation 4.

$$N = -\sigma \times \text{Sign}(U_s - 0.5) \times \log(1 - 2 \times |U_s - 0.5|) \quad (4)$$

- Calculating the perturbed sample $x_{\text{pert}} = x + n$.
- In the classification and privacy step, evaluating the perturbed sample x_{pert} using the target model f . It is ensured that the privacy budget is not exceeded by verifying the differential privacy requirement.
- Finally, if $f(x_{\text{pert}})$ misclassifies the perturbed sample and the differential privacy condition is satisfied, the adversarial example is generated ($x_{adv} = x_{\text{pert}}$).

Algorithm 1 presents a DP-NI attack. In this algorithm, a nested loop is used for adversarial sample generation. In this loop, for each batch, random uniform samples (U_s) are used to add randomness into the process (Line 4). Laplace noise is added into the original input data (x) to create a perturbed input (x_{adv}). The magnitude of the noise is determined by the previously calculated noise scale (σ) (Lines 5 and 6). A check ensures the perturbed input adheres to the specified differential privacy constraint. This involves evaluating the sensitivity, noise scale, and privacy budget. If the constraint is satisfied, the process continues; otherwise, the batch is skipped (Lines 7 - 14). If the privacy constraint is met, the perturbed input (x_{adv}) is fed into the target model (M). The model's outputs are then used to calculate a loss, which measures the dissimilarity between the predicted outputs and the real labels for the batch.

Gradients are backpropagated, and the model parameters are updated using an optimizer.

4.1.1. Toy Example:

Let's assume that the following vector is an example of the data set, for simplicity, we have considered only 30 features.

$$x = [1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0]$$

Let's assume the following parameters:

- ϵ (epsilon, privacy budget) = 1.0
- s (sensitivity) = 1.0
- η (learning rate) = 0.1
- Number of epochs (N_{ep}) = 1 (for simplicity)

So:

$$\sigma = \frac{s}{\epsilon} = \frac{1.0}{1.0} = 1.0$$

Then uniform samples U_s with the same shape as x will be:

$$U_s = [0.548, 0.715, 0.603, 0.544, 0.423, 0.645, 0.437, 0.891, 0.963, 0.383, 0.791, 0.528, 0.568, 0.925, 0.071, 0.087, 0.020, 0.832, 0.778, 0.870, 0.978, 0.799, 0.461, 0.780, 0.118, 0.639, 0.143, 0.944, 0.521, 0.414]$$

In next step Laplace noise N will compute as:

$$N = [0.10, 0.56, 0.23, 0.09, -0.16, 0.34, -0.13, 1.53, 2.62, -0.26, 0.87, 0.05, 0.14, 1.90, -1.95, -1.74, -2.20, 1.09, 0.81, 1.34, 3.15, 0.91, -0.08, 0.82, -1.44, 0.32, -1.24, 2.20, 0.04, -0.18]$$

And finally the adversarial sample x_{adv} is:

$$x_{adv} = [1.10, 1.56, 0.23, 0.09, -0.16, 1.34, 0.86, 1.53, 2.62, -0.26, 1.87, 0.05, 0.14, 2.90, -1.95, -1.74, -2.20, 2.09, 0.81, 1.34, 4.15, 1.91, -0.08, 0.82, -1.44, 0.32, -1.24, 3.20, 0.04, -0.18]$$

It should be noted that in this example, high values of privacy budget, sensitivity, and learning rate were used to ensure that the differences between the adversarial samples and the original samples are significant and clearly observable. Nonetheless, as can be seen, the generated values are close to 0 and 1.

Time Complexity: To analyze the time complexity of the *DP-NI Attack*, we consider its main components in Algorithm 1. The algorithm iterates over a fixed number of training epochs (N_{ep}) and processes data in batches. The time complexity for these iterations is $O(N_{ep} \cdot N \cdot B)$, where N is the total number of training samples, and B is the batch size. Similarly, the time complexity of generating random uniform samples, calculating Laplace noise, and checking the differential privacy constraint is $O(N_{ep} \cdot N \cdot B)$. Therefore, the overall time complexity of the *DP-NI Attack* algorithm is $O(N_{ep} \cdot N \cdot B)$. Given that N_{ep} and B are fixed numbers, the final time complexity for this algorithm simplifies to $O(N)$.

4.2. GDP Attack

In this sub-section, we propose differentially private gradient [29] for Model Poisoning (GDP) as an adversarial attack that leverages gradient-based differential privacy for model poisoning. In this method, we adapt the idea of using Gaussian noise to the gradients of the loss function during the training process to craft adversarial examples for malware classification

Algorithm 2 GDP Attack: Gradient-based Data Poisoning

Input: M :Model, $D = (x, y)$, ϵ :Privacy budget, s :Sensitivity

Output: x_{adv}

```

1: Initialize model parameters  $\theta$ 
2: for epoch  $\leftarrow 1$  to max_epochs do
3:   for  $i \leftarrow 1$  to  $N$  do
4:     Compute loss  $\mathcal{L}(\theta)$  using equation 6
5:     Compute gradients  $\nabla_{\theta} \mathcal{L}(\theta)$ 
6:     Generate Gaussian noise  $\mathcal{N}(0, \sigma^2 I)$ 
7:      $\tilde{g}_i \leftarrow \nabla_{\theta} \mathcal{L}(\theta) + \mathcal{N}(0, \sigma^2 I)$ 
8:      $\theta \leftarrow \theta - \alpha \cdot \tilde{g}_i$ 
9:   end for
10: end for
11: for each batch  $(x, y)$  in training data do
12:   if Check-Differential-Privacy( $x, s, \epsilon$ ) then
13:     if  $M(x) \neq y$  then
14:        $x_{adv} \leftarrow x$ 
15:     end if
16:   end if
17: end for
18: return  $x_{adv}$ 

```

tasks. The objective is to perturb the gradients using Gaussian noise to create adversarial examples that lead to misclassification. This method is presented in algorithm 2. In this algorithm, a nested loop is used for model poisoning. The key steps within this loop are as follows:

Consider a binary classification problem with training data like equation 5:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \quad (5)$$

where x_i is the feature vector and $y_i \in \{0, 1\}$ is the corresponding label. The goal is to learn a model $f_{\theta}(x)$ parameterized by θ that predicts the class label of each input x . The loss function for binary classification is the cross-entropy loss in equation (6) (Line 4):

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N y_i \log(f_{\theta}(x_i)) + (1 - y_i) \log(1 - f_{\theta}(x_i)) \quad (6)$$

To perform the differential privacy-based adversarial attack, we perturb the gradients of the loss function using Gaussian noise. The perturbed gradients $\tilde{g}_i$ for each training sample i are calculated as equation (7) (Lines 5 and 6):

$$\tilde{g}_i = \nabla_{\theta} \mathcal{L}(\theta) + \mathcal{N}(0, \sigma^2 I) \quad (7)$$

where $\mathcal{N}(0, \sigma^2 I)$ represents Gaussian noise with mean 0 and covariance matrix $\sigma^2 I$. We then update the model parameters using the perturbed gradients in equation (8) (Lines 7 and 8):

$$\theta \leftarrow \theta - \alpha \cdot \tilde{g}_i \quad (8)$$

where α is the learning rate.

By adding Gaussian noise to the gradients, we introduce randomness into the learning process, making it challenging for the learning algorithm to converge accurately. This randomness can lead to the model misclassifying inputs, thus demonstrating the potential of differential privacy-based adversarial attacks in the context of malware classification.

4.3. Toy Example:

Let's assume same vector and parameters(with DP-NI attack):
 $x = [1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0]$

- ϵ (epsilon, privacy budget) = 1.0
- δ (delta, a small value) = $1e-5$
- s (sensitivity) = 1.0
- η (learning rate) = 0.1
- Number of epochs (N_{ep}) = 1 (for simplicity)
- $\sigma = 4.8448$

The generated noise N is: $N = [4.18, -3.59, 10.99, -7.04, 0.22, -0.90, 7.42, 7.11, 0.75, 1.83, -4.30, -9.59, -1.68, 0.75, 5.96, 5.82, -1.87, -1.46, -5.08, -6.87, -8.26, 9.45, -3.46, -2.12, -6.06, 2.76, -7.81, -1.03, -4.33, 1.87]$

The adversarial sample x_{adv} is: $x_{adv} = [5.18, -2.59, 10.99, -7.04, 0.22, 0.09, 8.42, 7.11, 0.75, 1.83, -3.30, -9.59, -1.68, 1.75, 5.96, 5.82, -0.87, -0.46, -5.08, -6.87, -7.26, 10.45, -2.46, -2.12, -6.06, 3.76, -7.81, -0.03, -4.33, 1.87]$

Again, it should be noted that in this example, high values of privacy budget, sensitivity, and learning rate were used to ensure that the differences between the adversarial samples and the original samples are significant and clearly observable. Nonetheless, as can be seen, the generated values are close to 0 and 1.

Time Complexity: To analyze the time complexity of the *GDP Attack*, we consider its major components in Algorithm 2. The algorithm iterates over a fixed number of training epochs (max_epochs). The time complexity of these steps depends on the complexity of the loss function and the model's architecture, denoted as $f(\theta)$. The algorithm computes the loss $\mathcal{L}(\theta)$ and its gradients $\nabla_{\theta}\mathcal{L}(\theta)$, both of which also depend on $f(\theta)$. Therefore, the overall time complexity of the *GDP Attack* algorithm is $O(max_epochs \cdot N \cdot f(\theta))$.

5. Defense Method

We propose a defense method for binary classification tasks to defend against the differential privacy-based adversarial attacks described in the previous section. This defense involves a multi-pronged strategy, including objective modification, differential privacy noise incorporation, and privacy budget management. We aim to create a formidable shield against proposed adversarial attacks by amalgamating these components. This method is presented in the algorithm 3.

5.1. DP-NC defense

In this subsection, we propose Differential Privacy-Based Noise Clipping (DP-NC) as a defense method against proposed adversarial attacks. The basic concept of this defence approach is similar to adversarial training, with the main difference being that the created noise added by differential privacy techniques.

Algorithm 3 DP-NC Defense: Differential Privacy-Based Noise Clipping

Input: M :Model, $D = (x, y)$, ϵ :Privacy budget, s :Sensitivity

Output: M

```

1: Initialize model parameters  $\theta$ 
2: for epoch  $\leftarrow 1$  to  $max\_epochs$  do
3:   for  $i \leftarrow 1$  to  $N$  do
4:     Compute modified loss  $\mathcal{L}(\theta)$  using equation 9
5:     Compute gradients  $\nabla_{\theta}\mathcal{L}(\theta)$ 
6:      $laplace\_noise \sim Laplace(0, \frac{2}{\epsilon N})$ 
7:      $clipped\_noise \leftarrow Clip(laplace\_noise, \frac{B}{\epsilon})$ 
8:      $random\_noise \sim Laplace(0, \frac{B}{\epsilon})$ 
9:      $DP\_Noise \leftarrow clipped\_noise + random\_noise$ 
10:     $\tilde{g}_i \leftarrow \nabla_{\theta}\mathcal{L}(\theta) + DP\_Noise$ 
11:     $\theta \leftarrow \theta - \alpha \cdot \tilde{g}_i$ 
12:   end for
13: end for
14: return  $M$ 
```

We modify the binary classification training to handle perturbations. To do this, we use differential privacy to clip noises by changing the loss function. The modified loss function is defined in equation (9) (Lines 4-6):

$$\mathcal{L}_{DP}(\theta) = \mathcal{L}(\theta) + \frac{1}{N} \sum_{i=1}^N DP_Noise(\epsilon, \delta) \cdot \|\tilde{g}_i\|_2 \quad (9)$$

where $\mathcal{L}_{DP}(\theta)$ is the modified loss function, $DP_Noise(\epsilon, \delta)$ is a differentially private noise mechanism, and $\|\tilde{g}_i\|_2$ is the Euclidean norm of the perturbed gradient for training sample i . Then, we implement a differentially private noise mechanism to protect the gradients(model updates). So, we use Laplace noise for perturbed gradients as equation (10) (Lines 7-10):

$$\tilde{g}_i = \nabla_{\theta}\mathcal{L}(\theta) + Laplace(0, \frac{2}{\epsilon N}) \quad (10)$$

where $Laplace(0, \frac{2}{\epsilon N})$ represents Laplace noise with mean 0 and scale parameter $\frac{2}{\epsilon N}$. The next step is to manage the privacy budget (ϵ and δ) to control the trade-off between privacy and utility. We experiment with different privacy budgets to find an appropriate balance. Then, we utilize regularization techniques such as weight decay and hyper-parameter tuning to fine-tune the binary classification model's performance and privacy guarantees.

Time Complexity: To analyze the time complexity of the *DP-NC Defense*, we examine its primary operations as outlined in algorithm 3. The algorithm involves iterating over a fixed number of training epochs (max_epochs), and within each epoch, it iterates over the training samples (N). The time complexity for these iterations is $O(max_epochs \cdot N)$. For each sample, the algorithm computes the loss $\mathcal{L}(\theta)$, calculates gradients $\nabla_{\theta}\mathcal{L}(\theta)$, and generates Laplace noise $Laplace(0, \frac{2}{\epsilon N})$. The time complexity of these steps depends on the complexity of the loss function and the model's architecture, denoted as $O(f(\theta))$. Therefore, the overall time complexity of the *DP-NC Defense* algorithm is $O(max_epochs \cdot N \cdot f(\theta))$.

6. Simulation Setup

This section outlines the setup for conducting simulations, ensuring the reproducibility of our research findings. We describe the system configuration, datasets, parameter configurations, and used metrics.

6.1. System Setting

The simulation environment was set up using the Google Colab Notebook service. The system had the following characteristics: it ran on Python version 3.10.12, had a maximum RAM capacity of 12.7 GB within the Colab Notebook, and was powered by an Intel Xeon CPU.

6.2. Datasets

In this research, we conducted static analysis on *three* Android malware datasets. To our knowledge, permissions constitute a fundamental profile of an Android application (APK) file, providing vital information that the Android operating system carefully evaluates during the installation process [30]. We do experiments on 300 permission features extracted from Android samples.

Drebin dataset: The Drebin malware dataset is designed to support research in Android malware detection by providing a collection of malware samples and their features. This dataset contains 118,505 Android samples from various sources and 5000 samples in this dataset are malicious. It includes both benign and malicious applications, with extracted features that are used to differentiate between the two [31]. By extracting from this dataset, we can access different types of features, including:

- Permissions: Permissions requested by the applications (e.g., access to SMS, contacts).
- APIs: APIs used by the applications.
- Hardware Components: Information about the hardware components accessed by the applications.
- Intents: Intents used by the applications for inter-process communication.
- Network Addresses: Information about network addresses contacted by the applications.

Genome dataset: The Genome malware dataset was created by researchers from North Carolina State University and is widely used for studying and benchmarking Android malware detection methods. This dataset was developed to facilitate the study of Android malware and to provide a comprehensive collection of malicious applications for research purposes. The dataset contains approximately 1,200 malware samples collected between August 2010 and October 2011, which are categorized into different malware families based on their behavior and characteristics. Similar to Drebin malware datasets, features such as permissions, API calls, and other behavioral characteristics can be extracted from the samples [32, 33].

Contagio dataset: The Contagio malware dataset is a comprehensive collection of malware samples and related data curated

by the Contagio community. This dataset comprises 11,960 mobile malware samples and 16,800 benign samples and includes a wide range of malware types such as Trojans, ransomware, rootkits, spyware, and more. It covers multiple platforms, including Windows, Android, and macOS, making it useful for cross-platform malware analysis. Each malware sample is accompanied by metadata that may include information such as the type of malware, hash values (MD5, SHA1, SHA256), date of discovery, and additional notes on the malware's behavior or origin. Similar to Drebin and Contagio datasets, features such as permissions, API calls, and other behavioral characteristics can be extracted from each sample [34].

6.3. Machine learning metrics

In this subsection, we introduce the ML metrics [35, 36] used for comparing proposed methods. In a binary classification task, the confusion matrix is a 2x2 matrix that classifies samples into four categories, which we use to define other ML metrics, and we can define these categories for Android malware detection as follows:

- True Positives (TP): These are cases where the model correctly predicts the presence of malware.
- False Positives (FP): These are cases where the model inaccurately predicts the presence of malware.
- True Negatives (TN): These are cases where the model accurately predicts the absence of malware.
- False Negatives (FN): These are cases where the model inaccurately predicts the absence of malware.

We use a confusion matrix to calculate the following ML metrics used to assess the performance of classification models:

- Accuracy: provides an overall measure of how well the model distinguishes between malware and benign samples in the dataset. It is calculated using the equation (11):

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \quad (11)$$

- Precision: measures the accuracy of the positive predictions made by the model, indicating how reliable it is in identifying malware samples without falsely labeling benign samples as malicious. It is calculated using the equation (12):

$$\text{Precision} = \frac{TP}{TP + FP} \quad (12)$$

- Recall: measures the ability of the model to identify all the actual positive samples correctly, indicating how effectively it captures malicious samples without missing any. It is calculated using the equation (13):

$$\text{Recall} = \frac{TP}{TP + FN} \quad (13)$$

- False Positive Rate (FPR): The FPR indicates the model's tendency to misclassify benign applications as malicious. It is calculated using the equation (14):

$$FPR = \frac{FP}{FP + TN} \quad (14)$$

- F1-Score: considers both false positives and false negatives, making it particularly suitable for tasks where the class distribution is imbalanced or when both errors are critical. It is calculated using the equation (15):

$$F1\text{-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (15)$$

7. Experimental Results

This section presents the results of applying the proposed methods to Android datasets. Our analysis encompasses three distinct scenarios. The first scenario entails a situation where no attack has occurred against training data. In the second scenario, we investigate the case where adversarial attacks are launched on the data. Lastly, the third scenario encapsulates deploying the proposed defense mechanism against the proposed adversarial attacks.

7.1. Results of applying proposed method on Drebin Dataset

In Fig. 2 and Fig. 3, we present the accuracy and loss values resulting from applying the proposed methods to the Drebin dataset, utilizing 16762 samples for training each model. Fig. 2 shows the results for the DP-NI attack, while Fig. 3 present the results for the GDP attack. In both figures, the effects of the proposed DP-NC defense method against adversarial attacks are compared with scenarios where no attacks have occurred.

As depicted in Fig. 2 and Fig. 3, after 20 epochs of applying the deep neural network to classify samples in an attack-free scenario, the classification accuracy exceeds 91%. As depicted in Fig. 2, after the DP-NI attack, the classification accuracy notably drops to approximately 50%. Subsequently, after applying the proposed defense method, the classification accuracy rebounds, surpassing 72%. Investigation of the loss values reveals that a promising reduction trend in the attacked scenario is not observed in the loss value, suggesting potential challenges in achieving convergence. In Fig. 3, the results of applying the GDP attack and the DP-NC defense against it are presented. It can be seen that by using this type of attack, by increasing the number of epochs, the accuracy decreases, and this is completely justified considering that this attack changes the gradient. On the opposite side, the proposed defense has been able to improve the accuracy to a great extent and bring it to 70%.

In Table 2, the values of ML metrics after 20 epochs of training are presented. In the without-attack scenario, high values in the precision, recall, and F1-score, along with low FPR values, indicate the effective performance of the classification model. However, following DP-NI and GDP attacks, a substantial increase in FPR and a decrease in precision, recall, and F1-score are observed. Notably, the impact is more pronounced in the case of the GDP attack, underscoring its success. After

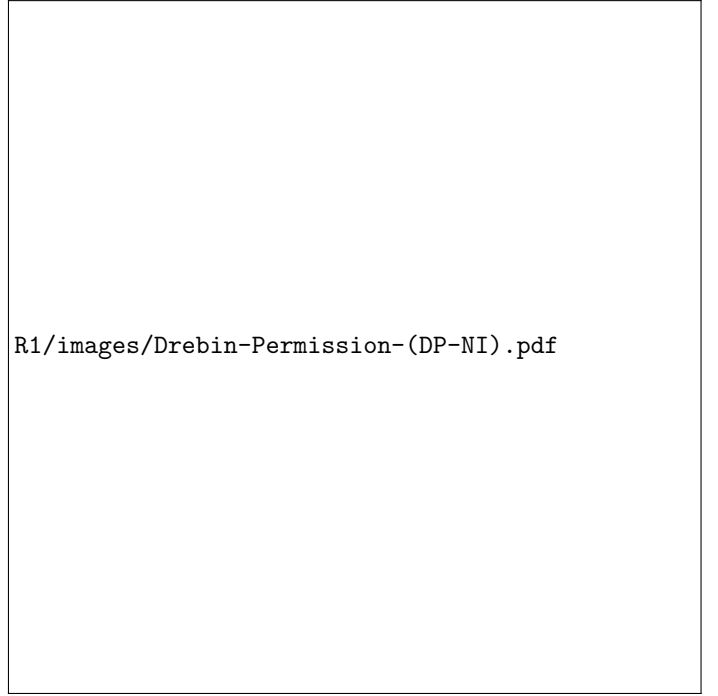


Figure 2: DP-NC defense against DP-NI Attack(Training on Drebin)



Figure 3: DP-NC defense against GDP Attack(Training on Drebin)

Table 2: ML Metric Values (Drebin, Training, epoch=20)

Scenario/Metric	Precision	Recall	FPR	F1.Score
No Attack	96.7	93.85	9.18	94.95
DP-NI Attack	61.44	54.43	54.08	57.72
DP-NC against DP-NI	84.44	73.56	28.29	78.63
GDP Attack	8.77	8.66	60.27	8.72
DP-NC against GDP	72.54	43.25	26.42	54.19

applying the DP-NC defense, a noticeable reduction in FPR is evident, accompanied by improvements in precision, recall, and F1-score. While these changes may not fully restore the system to its pre-attack state, they signify a degree of success for the proposed defense method.

Fig. 4 and Table 3 present the results of applying the trained model in various scenarios on the Drebin test dataset. Although a detailed investigation of the results in Table 3 reveals that the FPR associated with the DP-NI attack is notably higher, Fig. 4 shows that the GDP attack has been more successful in compromising the model, resulting in a significant reduction in accuracy. Considering other metrics, it can be concluded that a more significant number of samples are classified as FP in the GDP attack scenario.



Figure 4: Comparing Accuracy (Drebin Dataset-Test Data)

Table 3 shows that in the without-attack scenario, precision and recall values are 99.60% and 90.68%, respectively. This demonstrates a high number of correct sample classifications. Given the FPR value of 0.96%, it can be inferred that both TP and TN values are high, with only a few samples categorized as FP and FN.

The comparison of corresponding values for the two proposed attacks reveals a significant increase in FP and FN, partic-

ularly in the case of the GDP attack, where a substantial difference in precision and recall values is observed. Conversely, applying the proposed defense method against these attacks demonstrates that DP-NC contributes to a reduction in the number of FP and FN samples, improving overall results.

7.2. Results of applying proposed methods on Contagio dataset

Similar to the previous subsection, in Fig. 5 and Fig. 5, we present the accuracy and loss values resulting from the applying the proposed methods on the Contagio dataset, utilizing 13670 samples for model training. Fig. 5 illustrates the results for the DP-NI attack, while Fig. 5 showcases the results for the GDP attack. In both sub-figures, the impact of the proposed DP-NC defense method against the proposed adversarial attacks is compared with scenarios where no attacks have occurred.



Figure 5: DP-NC defense against DP-NI Attack(Training on Contagio)

As depicted in Fig. 5 and Fig. 6, after 20 epochs of applying the deep neural network to classify samples in without attack scenario, the classification accuracy exceeds 92%. In Fig. 5, after the DP-NI attack, the classification accuracy notably drops, reaching even lower than 50%. Subsequently, by

Table 3: ML Metric Values (Drebin, Test)

Scenario/Metric	Precision	Recall	FPR	F1.Score
No Attack	99.60	90.68	0.96	94.93
DP-NI Attack	70.08	50.03	56.96	58.38
DP-NC against DP-NI	99.10	71.83	11.50	83.29
GDP Attack	28.26	5.04	25.66	8.56
DP-NC against GDP	96.08	70.52	17.99	81.34

implementing the proposed defense method, the classification accuracy rebounds, surpassing 70%. Investigation of the loss values reveals that in the attacked scenario, there is not even a decreasing trend in the loss, suggesting challenges in achieving convergence.

Table 4 presents the metric values after training for 20 epochs on the Contagio dataset. High precision, recall, F1-score, and low FPR values in the without-attack scenario indicate that the classification model is working effectively. However, after DP-NI and GDP attacks, we notice a significant increase in FPR and a decrease in precision, recall, and F1-score. When we apply the DP-NC defense, we see a noticeable reduction in FPR and improvements in other metrics. Although these changes may not fully restore the system to its pre-attack state, they indicate a certain level of success for the proposed defense method.

R1/images/Contagio-Permission-(GDP).pdf

Figure 6: DP-NC defense against GDP Attack (Training on Contagio)

R1/images/Contagio-Accuracy-test.pdf

Figure 7: Comparing Accuracy (Contagio Dataset-Test Data)

Fig. 7 and Table 5 display the results of applying the trained model in various scenarios on the Contagio test dataset. Although a detailed investigation of the results in Table 5 indicates that the FPR associated with the DP-NI attack is notably higher, Fig. 7 makes it clear that the GDP attack has been more successful in compromising the model, resulting in a significant

Table 4: ML metrics Values (Contagio, Training , epoch=20)

Scenario/Metric	Precision	Recall	FPR	F1_Score
No Attack	97.85	95.56	16.40	96.69
DP-NI Attack	26.09	49.59	51.60	34.19
DP-NC against DP-NI	88.87	76.43	36.94	82.18
GDP Attack	62.75	20.32	60.86	30.70
DP-NC against GDP	83.82	96.58	34.35	89.75

Table 5: ML Metrics Values (Contagio, Test)

Scenario/Metric	Precision	Recall	FPR	F1_Score
No Attack	99.22	94.48	6.80	96.79
DP-NI Attack	94.39	66.48	36.64	78.01
DP-NI Attack	85.15	75.12	10.38	79.82
GDP Attack	14.59	34.44	25.17	20.50
DP-NC against GDP	45.51	90.71	14.74	60.61

reduction in accuracy.

With an FPR value of 6.80% in the no-attack scenario, it can be inferred that both TP and TN values are high, and only a few samples are categorized as FP and FN. The comparison of values for the two proposed attacks reveals a significant increase in FP and FN, particularly in the case of the GDP attack, where a substantial difference in precision and recall values is observed. Conversely, the application of the proposed defense method against these attacks demonstrates that DP-NC contributes to a reduction in the number of FP and FN samples, leading to an improvement in overall results.

7.3. Results of applying proposed method on Genome dataset

In Fig. 8 and Fig. 9, the accuracy and loss values of applying the proposed methods on the Genome dataset are presented. We used 13308 samples to train each model. In Fig. 8, the results for the DP-NI attack, and in Fig. 9, the results for the GDP attack are provided. In both sub-figures, after applying the proposed DP-NC defense method against the proposed adversarial attack, the corresponding results are compared with the scenario where no attack has occurred.

As depicted in Fig. 8 and Fig. 9, after 20 epochs of applying the deep neural network to classify samples in a scenario where no attack has occurred, the classification accuracy has reached above 93%. In Fig. 8, it can be seen that after the DP-NI attack, the classification accuracy has significantly decreased, reaching approximately 30%. Finally, the classification accuracy has increased by employing the proposed defense method, reaching above 65%. A glance at the loss values indicates that in the attacked scenario, there is not even a decreasing trend in the loss, suggesting that convergence may not be achieved. Similarly, Fig. 9 shows that after applying the GDP attack, the accuracy decreases to lower than 40%, and with the defense method, it increases to around 70%.

In Table 6, the ML metric values after 20 epochs training on the Genome dataset are presented. In the without-attack scenario, high precision, recall, F1-score, and low FPR values indi-

R1/images/Genome-Permission-(DP-NI).pdf

Figure 8: DP-NC defense against DP-NI Attack(Training on Genome)

Table 6: ML metrics Values (Genome, Training, epoch=20)

Scenario/Metric	Precision	Recall	FPR	F1_Score
No Attack	99.35	94.71	6.13	96.98
DP-NI Attack	94.03	31.44	51.93	47.12
DP-NC against DP-NI	98.35	56.70	21.49	71.93
GDP Attack	94.13	44.36	47.61	60.30
DP-NC against GDP	96.73	65.23	15.12	77.91

R1/images/Genome-Permission-(GDP).pdf

R1/images/Genome-Accuracy-test.pdf

Figure 9: DP-NC defense against GDP Attack(Training on Genome) cate the effective performance of the classification model. Upon applying the DP-NC defense, a noticeable reduction in FPR is evident, accompanied by improvements in precision, recall, and F1-score. While these changes may not fully restore the system to its pre-attack state, they signify a degree of success for the proposed defense method.

Fig. 10 and Table 7 present the results of applying the trained model in various scenarios on the Genome test dataset. Fig. 10 shows that the DP-NI attack has been more successful in compromising the model, resulting in a significant reduction in accuracy. Also, a detailed investigation of the results in Table 7 reveals that the FPR associated with the DP-NI attack is notably higher. Considering other metrics, it can be concluded that a greater number of samples are classified as FP in the DP-NI attack scenario.

Given the FPR value of 6.80%, it can be inferred that both TP and TN values are high, with only a minimal number of samples categorized as FP and FN. The comparison of corresponding values for the two proposed attacks reveals a significant increase in FP and FN, particularly in the case of the GDP attack, where a substantial difference in precision and recall values is observed. Finally, applying the proposed defense method against these attacks demonstrates that DP-NC contributes to

Figure 10: Comparing Accuracy (Genome Dataset-Test Data) a reduction in the number of FP and FN samples, improving overall results.

7.4. Comparison of Proposed Attacks with KFFPDet

In this subsection, we compare the results of the proposed attack methods, DP-NI and GDP, with a newly introduced technique called KFFPDet [37]. The objective of this comparison is to evaluate the effectiveness of the proposed methods against the KFFPDet approach across three datasets: Drebin, Contagio, and Genome.

Table 8 presents the results for training on the Drebin dataset. The DP-NI attack significantly impacts the model performance, reducing the accuracy to 52.13%, which indicates a substantial decrease from an no-attack scenario. In contrast, the GDP attack has a more severe impact, with an accuracy of only 27.27%. This indicates that the GDP attack is highly effective in disrupting the model, making it almost incapable of correctly identifying both positive and negative cases. KFFPDet, on the other hand, shows an accuracy of 44.40%, which is better than GDP but worse than DP-NI. Its precision is high at 94.03%, but this comes at the cost of recall, which is only 31.44%, leading to an F1-score of 47.12%. This suggests that KFFPDet might be focusing more on minimizing false positives, hence the high precision, but is less effective in identifying all positive instances.

Table 7: ML Metrics Values (Genome, Test)

Scenario/Metric	Precision	Recall	FPR	F1_Score
No Attack	99.80	95.03	2.36	97.35
DP-NI Attack	92.76	30.72	25.35	46.15
DP-NC against DP-NI	98.85	56.42	17.21	71.84
GDP Attack	92.12	41.47	24.01	57.20
DP-NC against GDP	98.38	65.54	12.31	78.67

Table 8: ML Metrics Values (Drebin, Train)

Scenario/Metric	accuracy	Precision	Recall	FPR	F1_Score
No Attack	92.96	96.7	93.85	9.18	94.95
DP-NI Attack	52.13	61.44	54.43	54.08	57.72
GDP Attack	27.27	8.77	8.66	60.27	8.72
KFFPDet [37]	44.40	94.03	31.44	51.93	47.12

Table 9: ML Metrics Values (Contagio, Train)

Scenario/Metric	accuracy	Precision	Recall	FPR	F1_Score
No Attack	94.21	97.85	95.56	16.40	96.69
DP-NI Attack	48.72	94.39	66.48	36.64	78.01
GDP Attack	34.89	14.59	34.44	25.17	20.50
KFFPDet [37]	53.84	34.76	60.55	49.06	44.17

Table 10: ML Metrics Values (Genome, Train)

Scenario/Metric	accuracy	Precision	Recall	FPR	F1_Score
No Attack	94.63	99.35	94.71	6.13	96.98
DP-NI Attack	32.06	94.03	31.44	51.93	47.12
GDP Attack	44.8	94.13	44.36	47.61	60.30
KFFPDet [37]	45.21	78.99	42.67	44.75	55.40

Table 9 details the results for training on the Contagio dataset. The DP-NI attack again shows a significant reduction in model performance, with an accuracy of 48.72%. The GDP attack exhibits a drastic performance degradation with an accuracy of 34.89%. This demonstrates that the GDP attack effectively compromises the model’s accuracy, making it unreliable. KFFPDet, with an accuracy of 53.84%, shows higher accuracy than both DP-NI and GDP attacks on this dataset. Its precision is lower at 34.76%, but its recall at 60.55% and F1-score at 44.17% reflect a more balanced degradation, suggesting that while it introduces more false positives than DP-NI, it maintains a higher rate of correctly identifying positive instances than GDP.

Table 10 summarizes the results for the Genome dataset. The DP-NI attack leads to an accuracy of 32.06%, and the GDP attack shows an accuracy of 44.8%, which is higher than DP-NI. Finally, KFFPDet, with an accuracy of 45.21%, performs higher than DP-NI and GDP in terms of accuracy. So, from an attacker perspective, DP-NI could outperform GDP and KFFPDet by decreasing the accuracy of the model.

In summary, by considering related results, the proposed DP-NI and GDP attacks show substantial effectiveness in de-

grading ML model performance. DP-NI tends to maintain higher precision but at the cost of recall, indicating fewer false positives but more false negatives. GDP, however, leads to a severe drop in all metrics, making it a highly disruptive attack. KFFPDet generally shows higher accuracy compared to GDP. While KFFPDet is effective, the proposed DP-NI attack often outperforms it in precision and specific scenarios, indicating that our proposed methods provide robust alternatives to existing attack techniques. The ability of DP-NI to maintain high precision could be particularly useful in scenarios where minimizing false positives is critical.

8. Discussion and Limitations

The exploration of adversarial threats to Android malware classifiers underscores the complexity of securing ML models in practical applications. The proposed DP-NI and GDP attacks reveal vulnerabilities in the classifier, with both attacks successfully compromising its accuracy. The findings emphasize the need for robust defenses in real-world deployment scenarios. Our proposed defense mechanism, DP-NC, showcases

promising results in mitigating the impact of adversarial attacks. By incorporating differential privacy, DP-NC enhances the model's robustness, leading to a significant recovery in accuracy. While the defense mechanism exhibits efficacy across the Drebin, Contagio, and Genome datasets, it is crucial to acknowledge variations in real-world scenarios. Dataset-specific characteristics may influence the generalizability of DP-NC across diverse datasets, necessitating further investigation into domain adaptation techniques for broader applicability. On the defense side, DP-NC requires additional computational resources during the training phase, which could impact its feasibility in resource-constrained environments. Balancing the trade-off between defense robustness and computational efficiency is a crucial consideration. The performance of the proposed defense mechanism, DP-NC, is sensitive to hyper-parameter tuning. Optimal configuration may vary across datasets and attack scenarios, posing a challenge in achieving a one-size-fits-all solution. Future work should explore automated approaches for hyper-parameter tuning to enhance the adaptability of DP-NC.

9. Conclusions and Future Work

In this paper, we have presented significant contributions to the fields of adversarial deep learning for Android malware detection. Our investigation into the utilization of differential privacy both by attackers to generate adversarial samples and by defense systems to mitigate adversarial attacks has yielded valuable insights. We proposed novel adversarial attack methods, namely the DP-NI and the GDP, leveraging the principles of differential privacy to craft adversarial samples while ensuring privacy protection. Moreover, we proposed a comprehensive defense mechanism, DP-NC, designed to mitigate the impact of differential privacy-based adversarial attacks on binary classification tasks. Our defense strategy incorporated differential privacy noise into gradient computation and applied regularization techniques to ensure privacy guarantees while maintaining model accuracy. Experimental evaluations on benchmark Android malware datasets validated proposed attacks and the robustness of our defense mechanism in mitigating the impact of attacks while preserving privacy and maintaining model accuracy. In conclusion, our findings underscore the importance of considering differential privacy principles in both attack and defense strategies for enhancing the robustness of deep learning models against adversarial attacks in Android malware detection.

Future research endeavors should focus on optimizing adversarial defense mechanisms like DP-NC. Exploring novel techniques for hyper-parameter tuning, model architecture adjustments, and fine-tuning training processes can enhance the generalizability and adaptability of defenses across diverse datasets and attack scenarios.

References

- [1] H. Bostani, V. Moonsamy, Evadedroid: A practical evasion attack on machine learning for black-box android malware detection, *Computers & Security* 139 (2024) 103676.
- [2] C. Gao, M. Cai, S. Yin, G. Huang, H. Li, W. Yuan, X. Luo, Obfuscation-resilient android malware analysis based on complementary features, *IEEE Transactions on Information Forensics and Security* (2023).
- [3] R. Taheri, M. Shojafar, M. Alazab, R. Tafazolli, Fed-iiot: A robust federated malware detection architecture in industrial iot, *IEEE transactions on industrial informatics* 17 (12) (2020) 8442–8452.
- [4] K. Lucas, S. Pai, W. Lin, L. Bauer, M. K. Reiter, M. Sharif, Adversarial training for {Raw-Binary} malware classifiers, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 1163–1180.
- [5] J. Kim, Y. Ban, E. Ko, H. Cho, J. H. Yi, Mapas: a practical deep learning-based android malware detection system, *International Journal of Information Security* 21 (4) (2022) 725–738.
- [6] S. Bhardwaj, A. S. Li, M. Dave, E. Bertino, Overcoming the lack of labeled data: Training malware detection models using adversarial domain adaptation, *Computers & Security* (2024) 103769.
- [7] S. Z. El Mestari, G. Lenzini, H. Demirci, Preserving data privacy in machine learning systems, *Computers & Security* 137 (2024) 103605.
- [8] S. Sajadmanesh, A. S. Shamsabadi, A. Bellet, D. Gatica-Perez, {GAP}: Differentially private graph neural networks with aggregation perturbation, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 3223–3240.
- [9] J. Giraldo, A. Cardenas, M. Kantarcioglu, J. Katz, Adversarial classification under differential privacy, in: *Network and Distributed Systems Security (NDSS) Symposium 2020*, 2020.
- [10] B. Xin, Y. Geng, T. Hu, S. Chen, W. Yang, S. Wang, L. Huang, Federated synthetic data generation with differential privacy, *Neurocomputing* 468 (2022) 1–10.
- [11] R. Taheri, Unveiling vulnerabilities in deep learning-based malware detection: Differential privacy driven adversarial attacks, <https://github.com/rahitaheri/Differential-Privacy-Driven-Adversarial-Attacks>, accessed: June 25, 2024 (2024).
- [12] H. Phan, M. T. Thai, H. Hu, R. Jin, T. Sun, D. Dou, Scalable differential privacy with certified robustness in adversarial learning, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 7683–7694.
- [13] M. Lecuyer, V. Atlidakis, R. Geambasu, D. Hsu, S. Jana, On the connection between differential privacy and adversarial robustness in machine learning, *stat* 1050 (2018) 9.
- [14] M. Lecuyer, V. Atlidakis, R. Geambasu, D. Hsu, S. Jana, Certified robustness to adversarial examples with differential privacy, in: *2019 IEEE symposium on security and privacy (SP)*, IEEE, 2019, pp. 656–672.
- [15] N. Tursynbek, A. Petiushko, I. Oseledets, Robustness threats of differential privacy, *arXiv preprint arXiv:2012.07828* (2020).
- [16] F. Boenisch, P. Sperl, K. Böttinger, Gradient masking and the underestimated robustness threats of differential privacy in deep learning, *arXiv preprint arXiv:2105.07985* (2021).
- [17] N. Phan, M. Vu, Y. Liu, R. Jin, D. Dou, X. Wu, M. T. Thai, Heterogeneous gaussian mechanism: Preserving differential privacy in deep learning with provable robustness, *arXiv preprint arXiv:1906.01444* (2019).
- [18] L. Zhu, X. Tang, M. Shen, F. Gao, J. Zhang, X. Du, Privacy-preserving machine learning training in iot aggregation scenarios, *IEEE Internet of Things Journal* 8 (15) (2021) 12106–12118.
- [19] D. Ye, S. Shen, T. Zhu, B. Liu, W. Zhou, One parameter defense—defending against data inference attacks via differential privacy, *IEEE Transactions on Information Forensics and Security* 17 (2022) 1466–1480.
- [20] Y. Wang, Q. Wang, L. Zhao, C. Wang, Differential privacy in deep learning: Privacy and beyond, *Future Generation Computer Systems* (2023).
- [21] R. Pinot, F. Yger, C. Gouy-Pailler, J. Atif, A unified view on differential privacy and robustness to adversarial examples, *arXiv preprint arXiv:1906.07982* (2019).
- [22] C. Liu, S. Chakraborty, P. Mittal, Dependence makes you vulnerable: Differential privacy under dependent tuples., in: *NDSS*, Vol. 16, 2016, pp. 21–24.
- [23] N. Ponomareva, H. Hazimeh, A. Kurakin, Z. Xu, C. Denison, H. B. McMahan, S. Vassilvitskii, S. Chien, A. G. Thakurta, How to dp-fy ml: A practical guide to machine learning with differential privacy, *Journal of Artificial Intelligence Research* 77 (2023) 1113–1201.
- [24] M. Nasr, S. Songi, A. Thakurta, N. Papernot, N. Carlin, Adversary instantiation: Lower bounds for differentially private machine learning, in: *2021 IEEE Symposium on security and privacy (SP)*, IEEE, 2021, pp. 866–882.

- [25] F. Wang, M. Xie, Z. Tan, Q. Li, C. Wang, Preserving differential privacy in deep learning based on feature relevance region segmentation, *IEEE Transactions on Emerging Topics in Computing* (2023).
- [26] T. T. Phuong, et al., Differentially private stochastic gradient descent via compression and memorization, *Journal of Systems Architecture* 135 (2023) 102819.
- [27] G. Muthukrishnan, S. Kalyani, Grafting laplace and gaussian distributions: A new noise mechanism for differential privacy, *IEEE Transactions on Information Forensics and Security* (2023).
- [28] M. Yang, T. Guo, T. Zhu, I. Tjuawinata, J. Zhao, K.-Y. Lam, Local differential privacy and its applications: A comprehensive survey, *Computer Standards & Interfaces* (2023) 103827.
- [29] L. Huang, J. Wu, D. Shi, S. Dey, L. Shi, Differential privacy in distributed optimization with gradient tracking, *IEEE Transactions on Automatic Control* (2024).
- [30] R. Taheri, R. Javidan, M. Shojafar, P. Vinod, M. Conti, Can machine learning model with static features be fooled: an adversarial machine learning approach, *Cluster computing* 23 (2020) 3233–3253.
- [31] D. Arp, M. Spreitzenbarth, H. Gascon, K. Rieck, C. Siemens, Drebin: Effective and explainable detection of android malware in your pocket., in: *Proc. of NDSS*, 2014.
- [32] Y. Zhou, X. Jiang, Dissecting android malware: Characterization and evolution, in: *2012 IEEE symposium on security and privacy*, IEEE, 2012, pp. 95–109.
- [33] Genome dataset, <http://http://www.malgenomeproject.org/>, [Online; accessed 15-Dec-2023] (2015).
- [34] Contagio dataset, <http://contagiomindump.blogspot.com/>, [Online; accessed 15-Dec-2023] (2019).
- [35] S. Gupta, K. Saluja, A. Goyal, A. Vajpayee, V. Tiwari, Comparing the performance of machine learning algorithms using estimated accuracy, *Measurement: Sensors* 24 (2022) 100432.
- [36] M. Golmaryami, R. Taheri, Z. Pooranian, M. Shojafar, P. Xiao, Setti: A self-supervised adversarial malware detection architecture in an iot environment, *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)* 18 (2s) (2022) 1–21.
- [37] K. Kong, L. Wang, Z. Zhang, Y. Li, D. Zhao, J. Huang, Kfpdet: Android malicious application detection system with assisted detection of adversarial samples, *Expert Systems with Applications* 252 (2024) 124095.

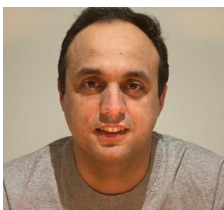


Alexander Gegov received the Ph.D. degree in control systems from the Bulgarian Academy of Sciences. He is an Associate Professor with the School of Computing, University of Portsmouth, UK. He is a recipient of the National Award for the Best Young Researchers. He has been a Humboldt Guest Researcher with the University of Duisburg-Essen, Germany, and an EU Visiting Researcher at the Delft University of Technology, The Netherlands.

Biographies



Rahim Taheri received his Ph.D. degree in Information Technology from Shiraz University of Technology, Iran, in 2020. Now he is a Lecturer in Cyber Security and Forensics at the University of Portsmouth, UK. Before joining the University of Portsmouth, he was a post-doctoral research associate at King's Communications, Learning, and Information Processing (Kclip) Lab, King's College London, UK. His main research interests include ML applications in security, adversarial ML, and federated learning.



Mohammad Shojafar is a Senior Lecturer (Associate Professor) in network security and an Intel Innovator, a Professional ACM member and ACM Distinguished Speaker, a Fellow of the Higher Education Academy, and a Marie Curie Alumni, working in the 5G & 6G Innovation Centre (5GIC & 6GIC), Institute for Communication Systems (ICS), at the University of Surrey, UK. He received his PhD in ICT from Sapienza University of Rome, Rome, Italy, in 2016 with an "Excellent" degree. He is an Associate Editor in *IEEE Transactions on Network and Service Management*, *IEEE Transactions on Intelligent Transportation Systems*, and *Computer Networks*. Dr Mohammad secured around £1.2M as PI in various EU/UK projects.



Farzad Arabikhan received his Ph.D. degree in Computer science from University of Portsmouth, UK, in 2017. He gained an MSc in Transportation Systems and Planning at the Sharif University of Technology, Iran. Now he is a Senior Lecturer at the University of Portsmouth, UK. His main research interests include Fuzzy Logic, Rule-Based Systems, Neural Networks and Genetic algorithm, Intelligent Transportation Systems and Activity Based Transport Modelling.

Fig. 1. DNN architecture.

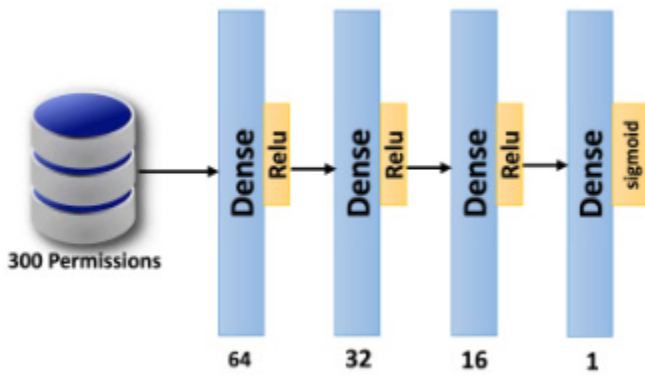


Fig. 2. DP-NC defense against DP-NI attack (Training on Drebin).

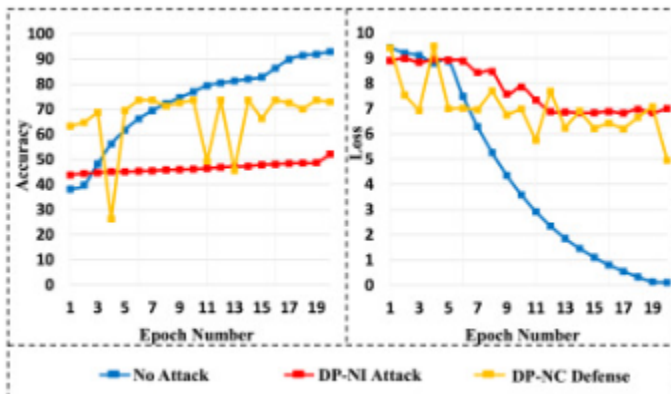


Fig. 3. DP-NC defense against GDP attack (Training on Drebin).

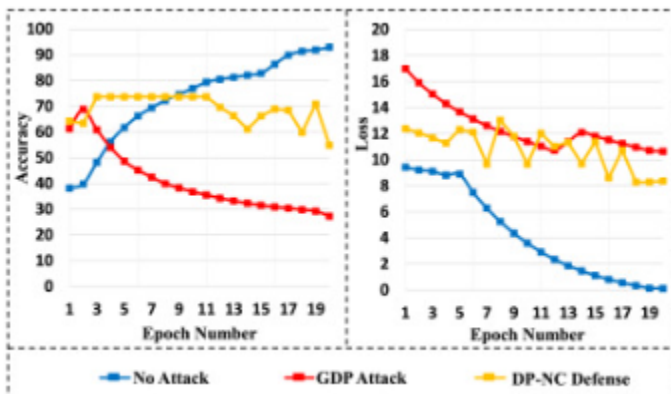


Fig. 4. Comparing accuracy (Drebin dataset-test data).

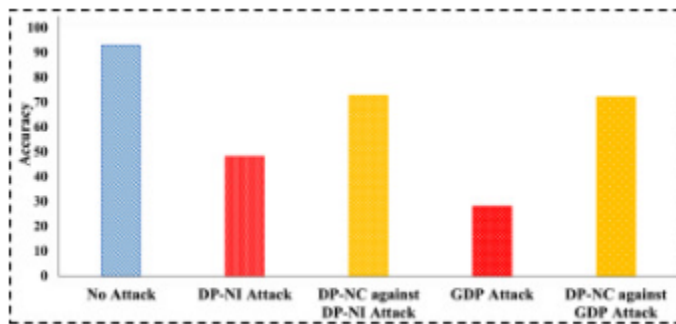


Fig. 5. DP-NC defense against DP-NI attack (Training on Contagio).

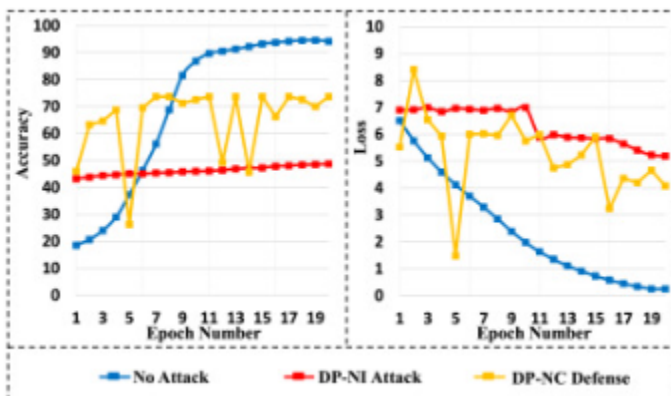


Fig. 6. DP-NC defense against GDP attack (Training on Contagio).

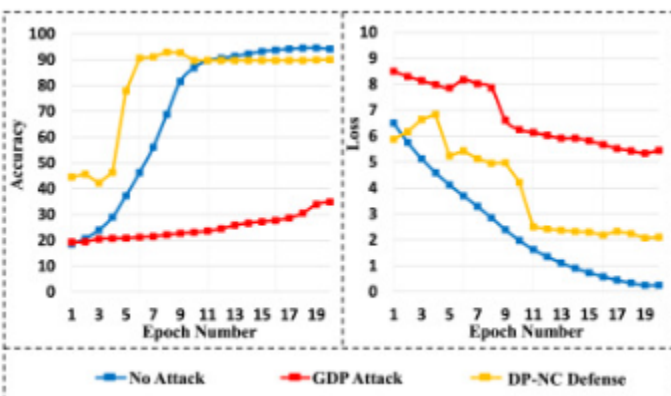


Fig. 7. Comparing accuracy (Contagio dataset-test data).

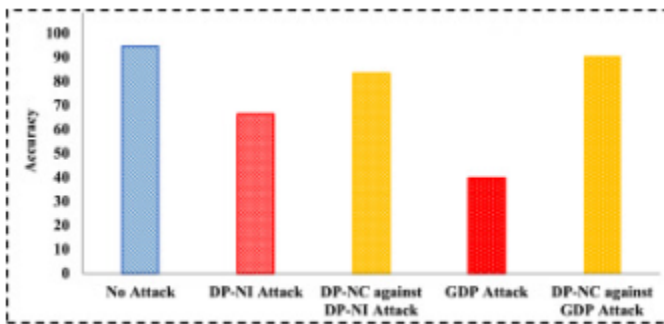


Fig. 8. DP-NC defense against DP-NI attack (Training on Genome).

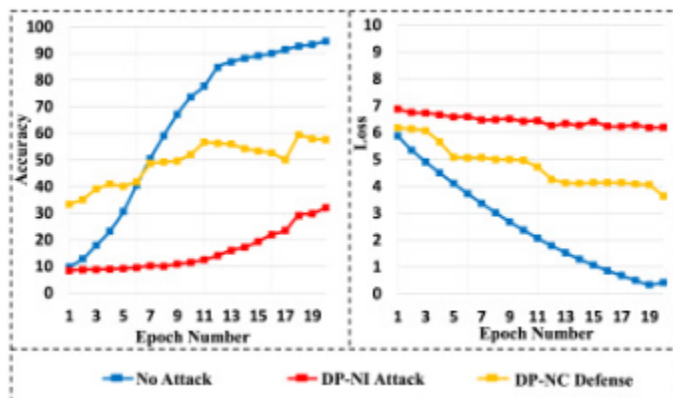


Fig. 9. DP-NC defense against GDP attack (Training on Genome).

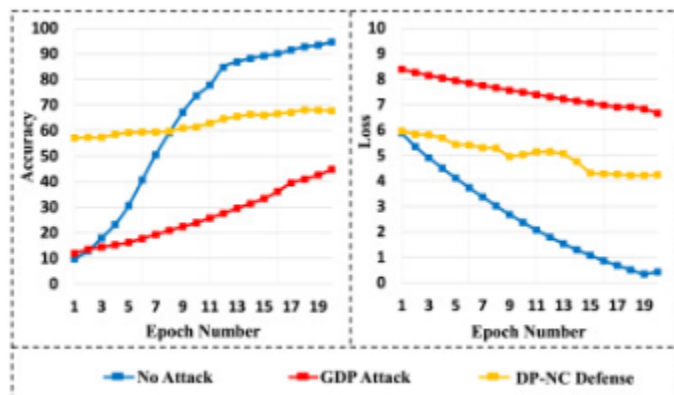


Fig. 10. Comparing accuracy (Genome dataset-test data).

